

Vena – Stroke Risk Prediction Pipeline

Enes Erbap & Sude Hatkaoğlu

Vena is a fully reproducible, production-ready MLOps pipeline that predicts stroke risk from healthcare data, even with extreme class imbalance.

Data, models, and code are all versioned and automated.

Our Engineering Stack

Package management: `uv` is Rust-based ultra-fast package manager

Code quality & typing: `Ruff` helps prevent bugs before the code runs.

Configuration: `Hydra` manages hyperparameters dynamically through `config.yaml`.

CLI interface: `Typer` creates a clean, user-friendly command line interface.

Containerization: `Docker` keeps environments independent with multi-layer caching.



01 SETUP – Architectural Foundation

Standardization:

Built using **Cookiecutter** MLOps templates.

Modular design:

Data processing, model architecture, and training loops are kept separate in modules such as `data.py` and `train.py`.

Environment isolation:

`uv.lock` and Docker keep development environments consistent across machines.

Zero hardcoding:

Logic and configuration are kept fully separate.

02 DATA – Engineering & Versioning

01

The challenge:

Extreme class imbalance: about 5% of cases are strokes.

02

Preprocessing:

Use median imputation for missing values such as BMI, One-Hot and Ordinal encoding for categorical variables, and `StandardScaler` to normalize features.

03

Stratified splitting:

Stroke cases are evenly distributed across the train and test sets.

04

Versioning:

`DVC` tracks large CSV files separately from Git through a local remote.

03 MODEL – Training & Tracking

Architecture:

Custom PyTorch multi-layer perceptron, StrokeMLP, with dimensions set dynamically.

Healthcare-ready metrics:

F1 Score, Recall, and AUROC are used instead of misleading accuracy.

Experiment tracking:

MLflow records hyperparameters, validation metrics, and the .pt2 model artifact for each run.

Optimization:

Automated early stopping helps prevent overfitting.

CURRENT STAGE: WE ARE NOT DEPLOYED YET.

04 DEPLOY – CI/CD & Production

01

Automated checks:

GitHub Actions will automate code linting, type-checking, and tests on every commit.

02

Model serving:

We will expose the PyTorch model through a FastAPI REST API for real-time inference.

03

Deployment strategy:

We plan to containerize the backend API and deploy it to a cloud platform such as Render or Hugging Face Spaces.

04

Frontend:

We plan to host the frontend on Vercel.

05 MONITOR

Planned monitoring stack

Evidently

Evidently: Detects data and concept drift in production to ensure model reliability.

Prometheus

Monitors FastAPI response time, VDS RAM usage, and system health to find slowdowns or failures.

Future scope: Automated retraining loops that start after verified data drift.

The Key Insight

“ML engineering is 80% of the work. A model without a pipeline is a science experiment. A model with a pipeline is a product.”

Thanks for listening.